



WPI

Lecture 2b – Linear Regression



Linear Model

- Model $f(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

- Linear Regression with least square error (LSE)

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$$

$$f(x_i) = wx_i + b, \text{ s.t. } f(x_i) \simeq y_i$$

$$\begin{aligned} (w^*, b^*) &= \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2 \\ &= \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2 \end{aligned}$$

Parameter Estimation: Closed Form Solution

$$E_{(w,b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$$

$$\frac{\partial E_{(w,b)}}{\partial w} = 2 \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b) x_i \right) ,$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2 \left(mb - \sum_{i=1}^m (y_i - wx_i) \right) ,$$

$$w = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} \left(\sum_{i=1}^m x_i \right)^2} , \quad b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i) , \quad \bar{x} = \frac{1}{m} \sum_{i=1}^m x_i$$

Multivariable Linear Regression

$$f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b, \quad \hat{\mathbf{w}} = (\mathbf{w}; b)$$

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1d} & 1 \\ x_{21} & x_{22} & \dots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{pmatrix}$$

$$\mathbf{y} = (y_1; y_2; \dots; y_m)$$

$$\hat{\mathbf{w}}^* = \arg \min_{\hat{\mathbf{w}}} (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$$

$$\frac{\partial E_{\hat{\mathbf{w}}}}{\partial \hat{\mathbf{w}}} = 2 \mathbf{X}^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}) \quad \hat{\mathbf{w}}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Linear Regression Model $f(\hat{\mathbf{x}}_i) = \hat{\mathbf{x}}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

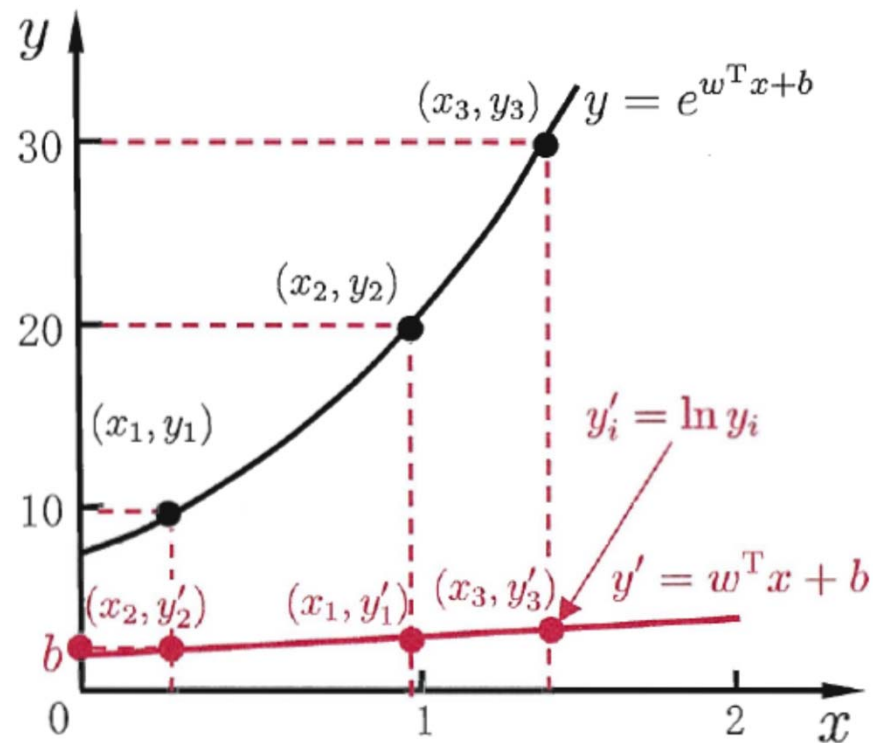
Log Linear Regression

- Linear regression

$$y = w^T x + b$$

- Log-linear regression

$$\ln y = w^T x + b$$



Use nonlinear functions in linear regression

- The functions $f(x)$ might be in any form including nonlinear functions or polynomials.
- The linearity, in the linear regression models, refers to the linearity of the coefficients β_k . That is, y is a linear function of the coefficients β_k .

$$y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{3i} + \varepsilon_i$$

$$y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{1i}^3 + \beta_4 X_{2i}^2 + \varepsilon_i$$

$$y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \beta_3 X_{1i} X_{2i} + \beta_4 \log X_{3i} + \varepsilon_i$$

- Those are not linear regression models

$$\log y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \varepsilon_i$$

$$y_i = \beta_0 + \beta_1 X_{1i} + \frac{1}{\beta_2 X_{2i}} + e^{\beta_3 X_{1i} X_{2i}} + \varepsilon_i$$

(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0}^{m-1}$, $b := 0$
 2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $err := (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$
 $b := b + err$
-
1. Initialize $\mathbf{w} := \mathbf{0}^{m-1}$, $b := 0$
 2. For every training epoch:
 - A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$
 - (a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$
 - (b) $\nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$
 $\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$
 - (c) $\mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$
 $b := b + \eta \times \underbrace{(-\nabla_b \mathcal{L})}_{\substack{\uparrow \\ \text{negative gradient}}}$
- learning rate $\nearrow$

(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode:

Vectorized

1. Initialize $\mathbf{w} := 0^{m-1}$, $\mathbf{b} := 0$
2. For every training epoch:

A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$

$$(a) \quad \hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$$

$$(b) \quad \nabla_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]}$$

$$\nabla_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]})$$

$$(c) \quad \mathbf{w} := \mathbf{w} + \eta \times (-\nabla_{\mathbf{w}} \mathcal{L})$$

$$b := b + \eta \times (-\nabla_b \mathcal{L})$$

learning rate

negative gradient

For-Loop

1. Initialize $\mathbf{w} := 0^{m-1}$, $\mathbf{b} := 0$
2. For every training epoch:

A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$

$$(a) \quad \hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$$

B. For weight j in $\{1, \dots, m\}$:

$$(b) \quad \frac{\partial \mathcal{L}}{\partial w_j} = -(y^{[i]} - \hat{y}^{[i]}) x_j^{[i]}$$

$$(c) \quad w_j := w_j + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial w_j}\right)$$

$$C. \quad \frac{\partial \mathcal{L}}{\partial b} = -(y^{[i]} - \hat{y}^{[i]})$$
$$b := b + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial b}\right)$$

(Least-Squares) Linear Regression

The update rule turns out to be this:

"On-line" mode

1. Initialize $\mathbf{w} := \mathbf{0}^{m-1}$, $b := 0$

2. For every training epoch:

A. For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$

(a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T} \mathbf{w} + b)$

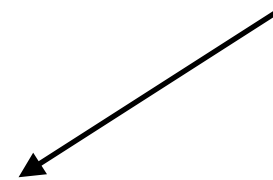
B. For weight j in $\{1, \dots, m\}$:

(b) $\frac{\partial \mathcal{L}}{\partial w_j} = -(y^{[i]} - \hat{y}^{[i]}) x_j^{[i]}$

(c) $w_j := w_j + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial w_j}\right)$

C. $\frac{\partial \mathcal{L}}{\partial b} = -(y^{[i]} - \hat{y}^{[i]})$
 $b := b + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial b}\right)$

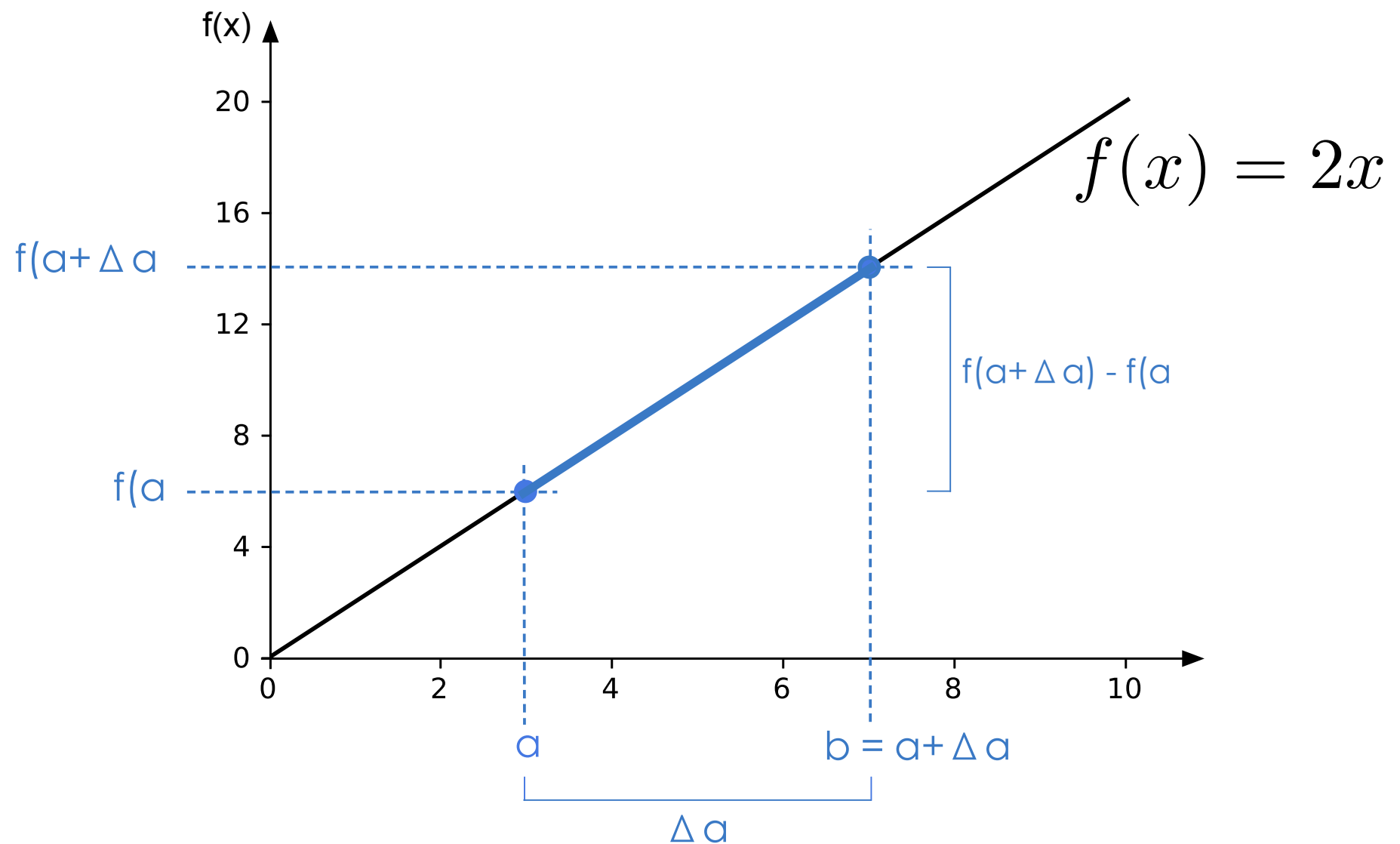
Coincidentally, this appears almost to be the same as the perceptron rule, except that the prediction is a real number and we have a learning rate



**First, let's briefly cover relevant background
info ...**

Differential Calculus Refresher

Derivative of a function = "rate of change" = "slope"



$$\text{Slope} = \frac{f(a + \Delta a) - f(a)}{a + \Delta a - a} = \frac{f(a + \Delta a) - f(a)}{\Delta a}$$

Function Derivative

$$f'(x) = \frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Example 1: $f(x) = 2x$

$$\begin{aligned} \frac{df}{dx} &= \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \frac{2x + 2\Delta x - 2x}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \frac{2\Delta x}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} 2. \end{aligned}$$

Numerical vs Analytical/Symbolical Derivatives

$$f'(x) = \frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

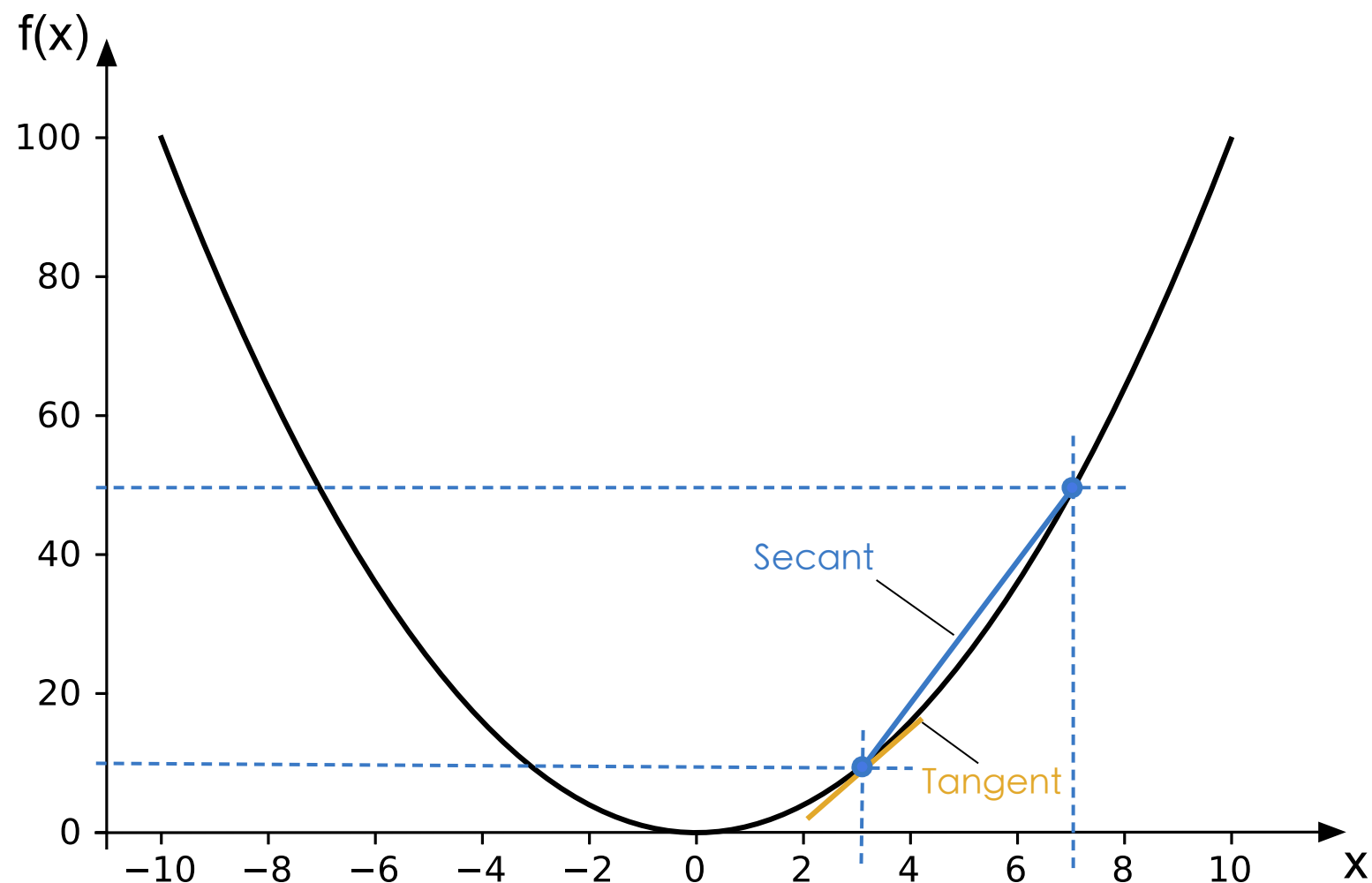
Example 2: $f(x) = x^2$

$$\begin{aligned} \frac{df}{dx} &= \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \frac{x^2 + 2x\Delta x + (\Delta x)^2 - x^2}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} \frac{2x\Delta x + (\Delta x)^2}{\Delta x} \\ &= \lim_{\Delta x \rightarrow 0} 2x + \Delta x. \end{aligned}$$

Numerical vs Analytical/Symbolical Derivatives

Conceptually, we obtained the derivative $\frac{d}{dx}x^2 = 2x$

By approximating the slope (tangent) by a second between two points (as before)



A Cheatsheet for You (1)

	Function $f(x)$	Derivative with respect to x
1	a	0
2	x	1
3	ax	a
4	x^2	$2x$
5	x^a	ax^{a-1}
6	a^x	$\log(a)a^x$
7	$\log(x)$	$1/x$
8	$\log_a(x)$	$1/(x \log(a))$
9	$\sin(x)$	$\cos(x)$
10	$\cos(x)$	$-\sin(x)$
11	$\tan(x)$	$\sec^2(x)$

Table D.1: Derivatives of common functions.

A Cheatsheet for You (2)

	Function	Derivative
Sum Rule	$f(x) + g(x)$	$f'(x) + g'(x)$
Difference Rule	$f(x) - g(x)$	$f'(x) - g'(x)$
Product Rule	$f(x)g(x)$	$f'(x)g(x) + f(x)g'(x)$
Quotient Rule	$f(x)/g(x)$	$[g(x)f'(x) - f(x)g'(x)]/[g(x)]^2$
Reciprocal Rule	$1/f(x)$	$-[f'(x)]/[f(x)]^2$
Chain Rule	$f(g(x))$	$f'(g(x))g'(x)$

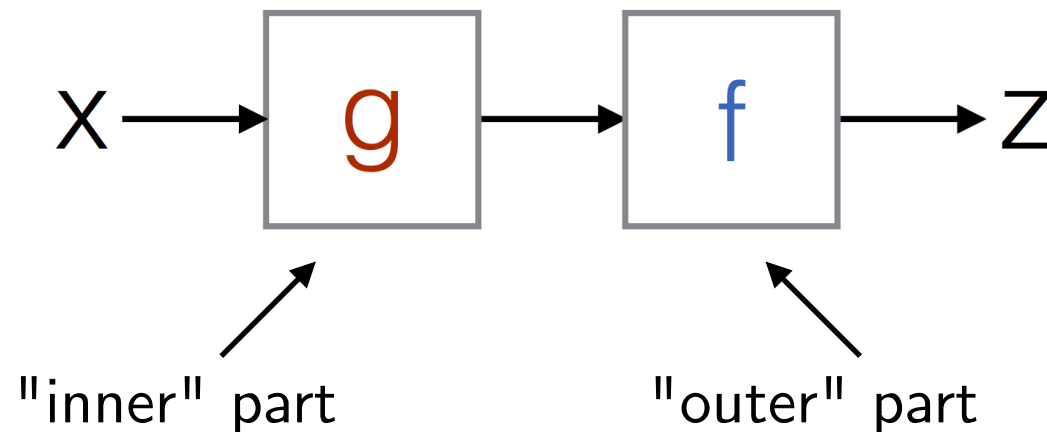
Chain Rule

- The chain rule is basically the essence of training (deep) neural networks
- If you understand and learn how to apply the chain rule to various function decompositions, deep learning will be super easy and even seem trivial to you from now on
- In fact, neural networks will become even easier to understand than any algorithm you learned about in my previous ML class

Chain Rule & "Computation Graph" Intuition

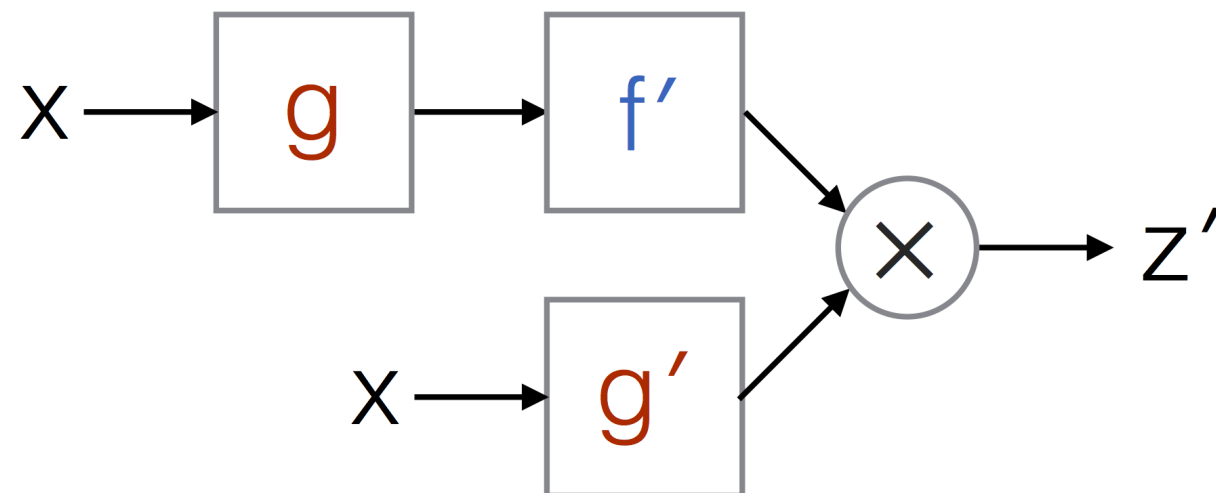
$$F(x) = f(g(x)) = z$$

Decomposition of some
(nested) function:



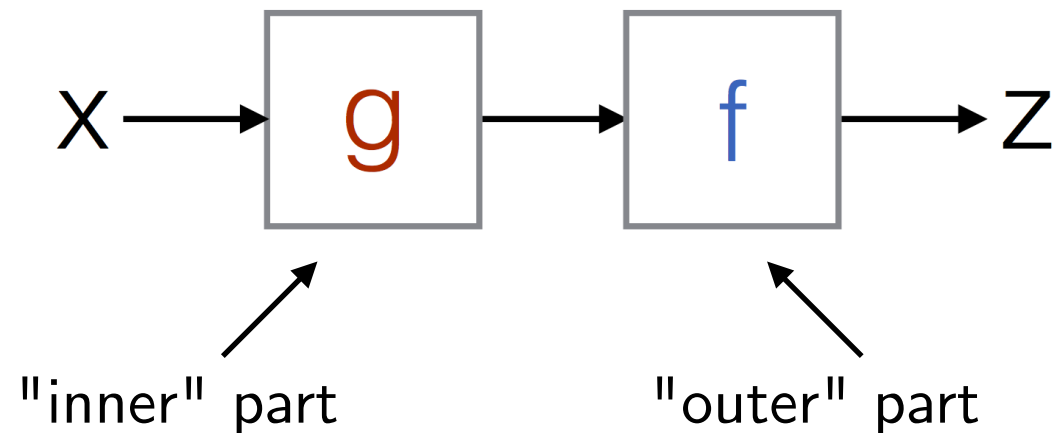
$$F'(x) = f'(g(x)) g'(x) = z'$$

Derivative of that nested
function:



Chain Rule & "Computation Graph" Intuition

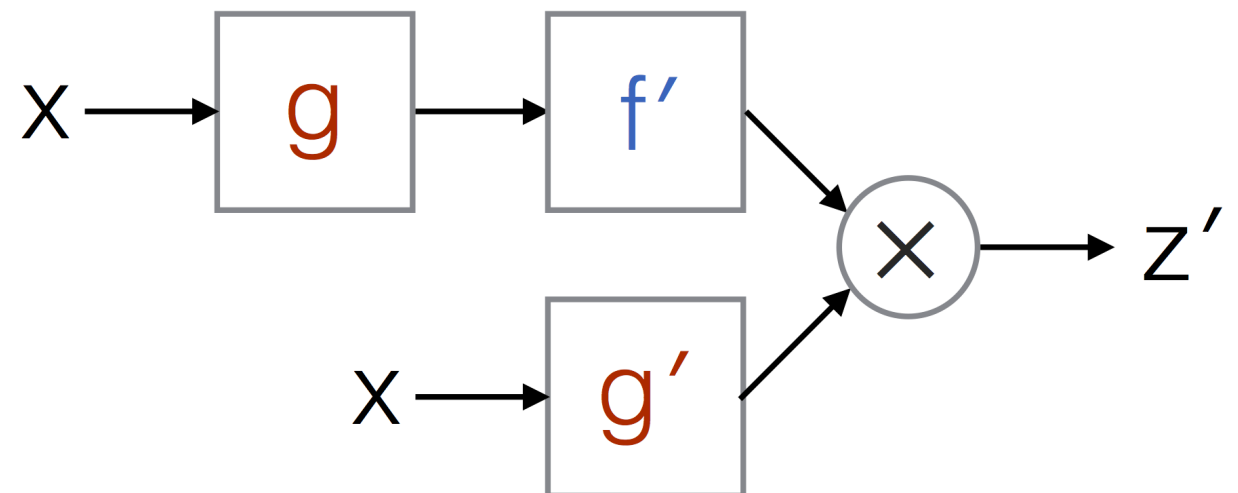
$$F(x) = f(g(x)) = z$$



Later, we will see that PyTorch can do that automatically for us :) (PyTorch literally keeps a computation graph in the background)

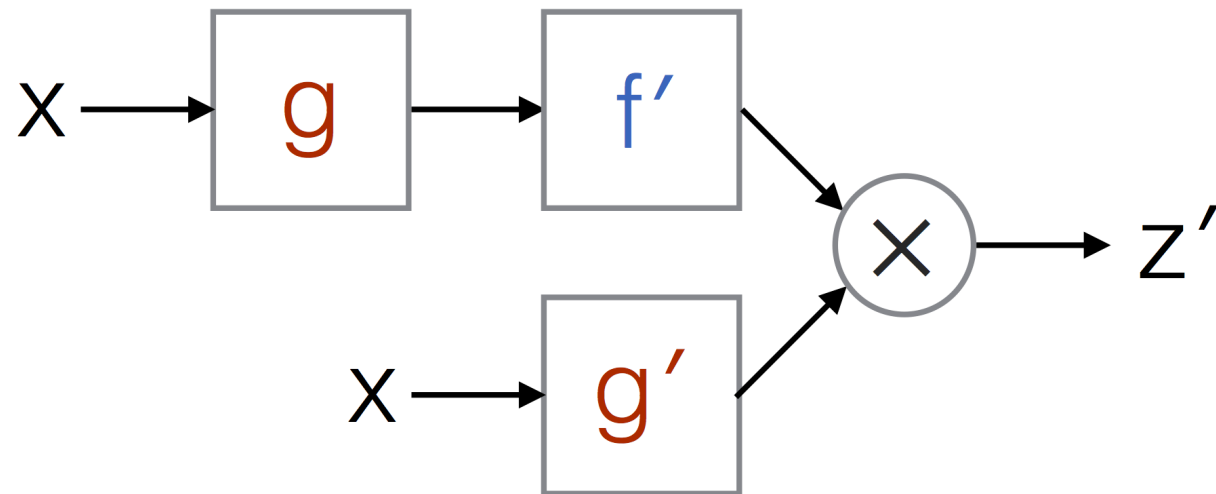
$$F'(x) = f'(g(x)) g'(x) = z'$$

Also, PyTorch can compute the derivatives of most (differentiable) functions automatically



Chain Rule & "Computation Graph" Intuition

$$F'(x) = f'(g(x)) g'(x) = z'$$



In text, for efficiency, we will mostly use the Leibniz notation:

$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Chain Rule Example

$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Example:

$$f(x) = \log(\sqrt{x})$$

substituting $\frac{df}{dx} = \frac{d}{dg} \log(g) \cdot \frac{d}{dx} \sqrt{x}$

with $\frac{d}{dg} \log(g) = \frac{1}{g} = \frac{1}{\sqrt{x}}$ and $\frac{d}{dx} x^{1/2} = \frac{1}{2} x^{-1/2} = \frac{1}{2\sqrt{x}}$

leads us to the solution $\frac{df}{dx} = \frac{1}{\sqrt{x}} \cdot \frac{1}{2\sqrt{x}} = \frac{1}{2x}$

Chain Rule for Arbitrarily Long Function Compositions

$$F(x) = f(g(h(u(v(x)))))$$

$$\begin{aligned}\frac{dF}{dx} &= \frac{d}{dx}F(x) = \frac{d}{dx}f(g(h(u(v(x))))) \\ &= \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}\end{aligned}$$

Chain Rule for Arbitrarily Long Function Compositions

$$\begin{aligned}\frac{dF}{dx} &= \frac{d}{dx} F(x) = \frac{d}{dx} f(g(h(u(v(x))))) \\ &= \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}\end{aligned}$$

Also called "reverse mode" as we start with the outer function. In neural nets, this will be from right to left.

We could also start from the inner parts ("forward mode")

$$\frac{dv}{dx} \cdot \frac{du}{dv} \cdot \frac{dh}{du} \cdot \frac{dg}{dh} \cdot \frac{df}{dg}$$

- Backpropagation (covered later) is basically "reverse" mode auto-differentiation
- It is cheaper than forward mode if we work with gradients, since then we have matrix-"vector" multiplications instead of matrix multiplications

Gradients: Derivatives of Multivariable* Functions

*note that in some fields, the terms "multivariable" and "multivariate" are used interchangeably, but here, we really mean "multivariable" because "multivariate" means "multiple outputs", which is not the case here -- similarly, in most DL applications output one prediction value, or one prediction value per training example

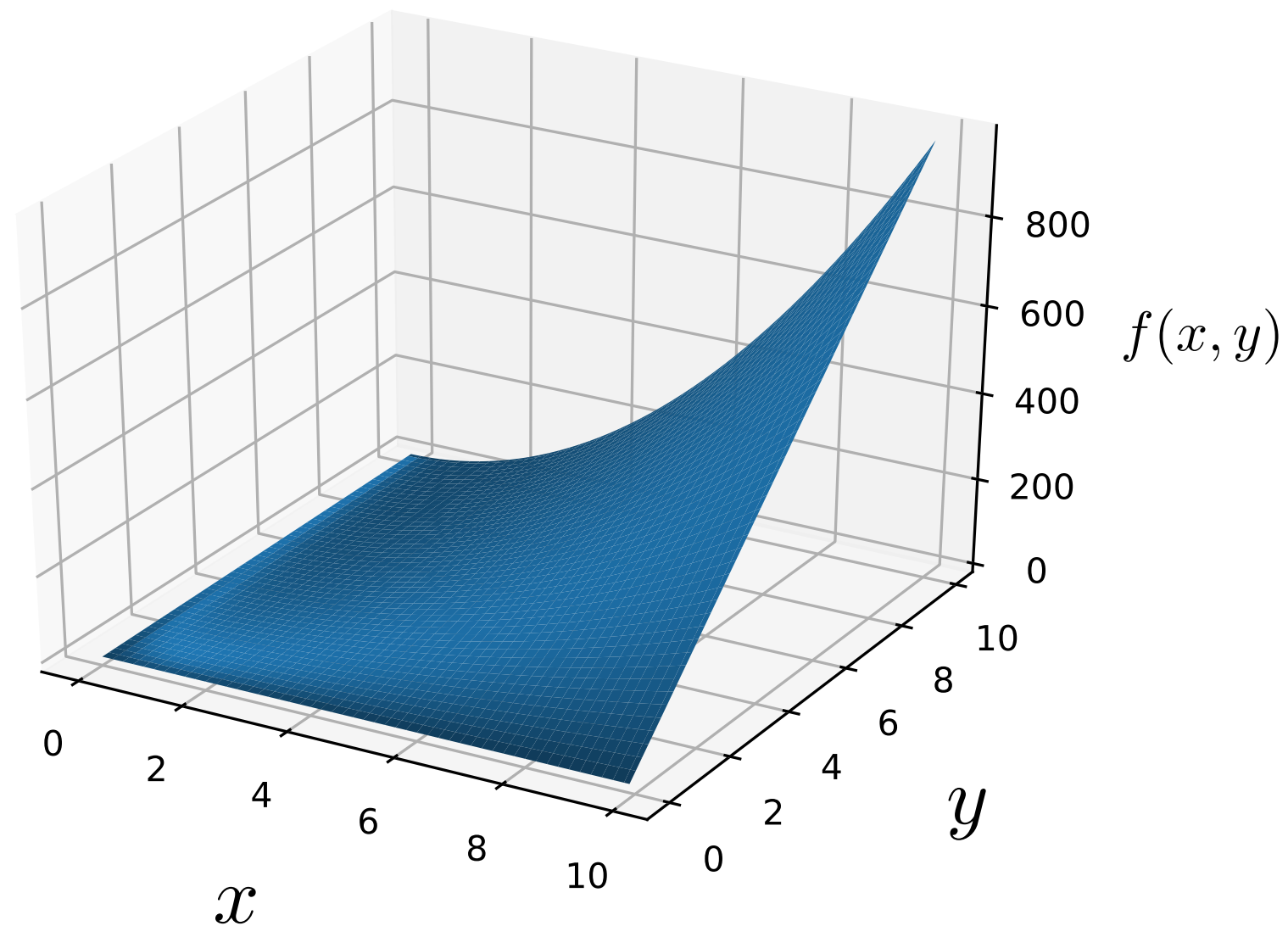
$$f(x, y, z, \dots)$$

$$\nabla f = \begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \\ \partial f / \partial z \\ \vdots \end{bmatrix}$$


For gradients, we use the "partial" symbol to denote partial derivatives; more of a notational convention and the concept is the same as before when we were computing ordinary derivatives (denoted them as "d")

Gradients: Derivatives of Multivariable Functions

Example: $f(x, y) = x^2y + y$



Gradients: Derivatives of Multivariable Functions

Example: $f(x, y) = x^2y + y$

$$\nabla f(x, y) = \begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \end{bmatrix},$$

where

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x} x^2y + y = 2xy$$

(via the power rule and constant rule), and

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y} x^2y + y = x^2 + 1.$$

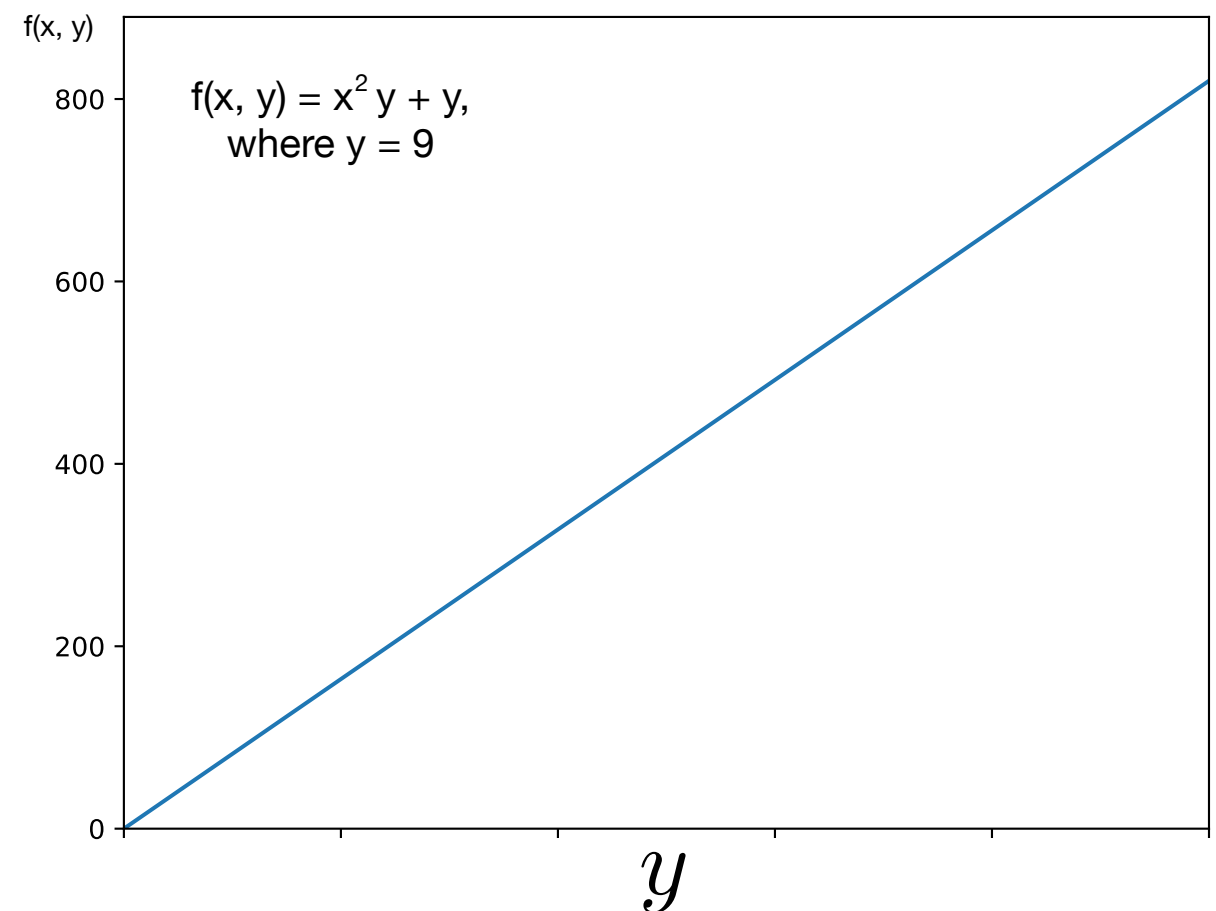
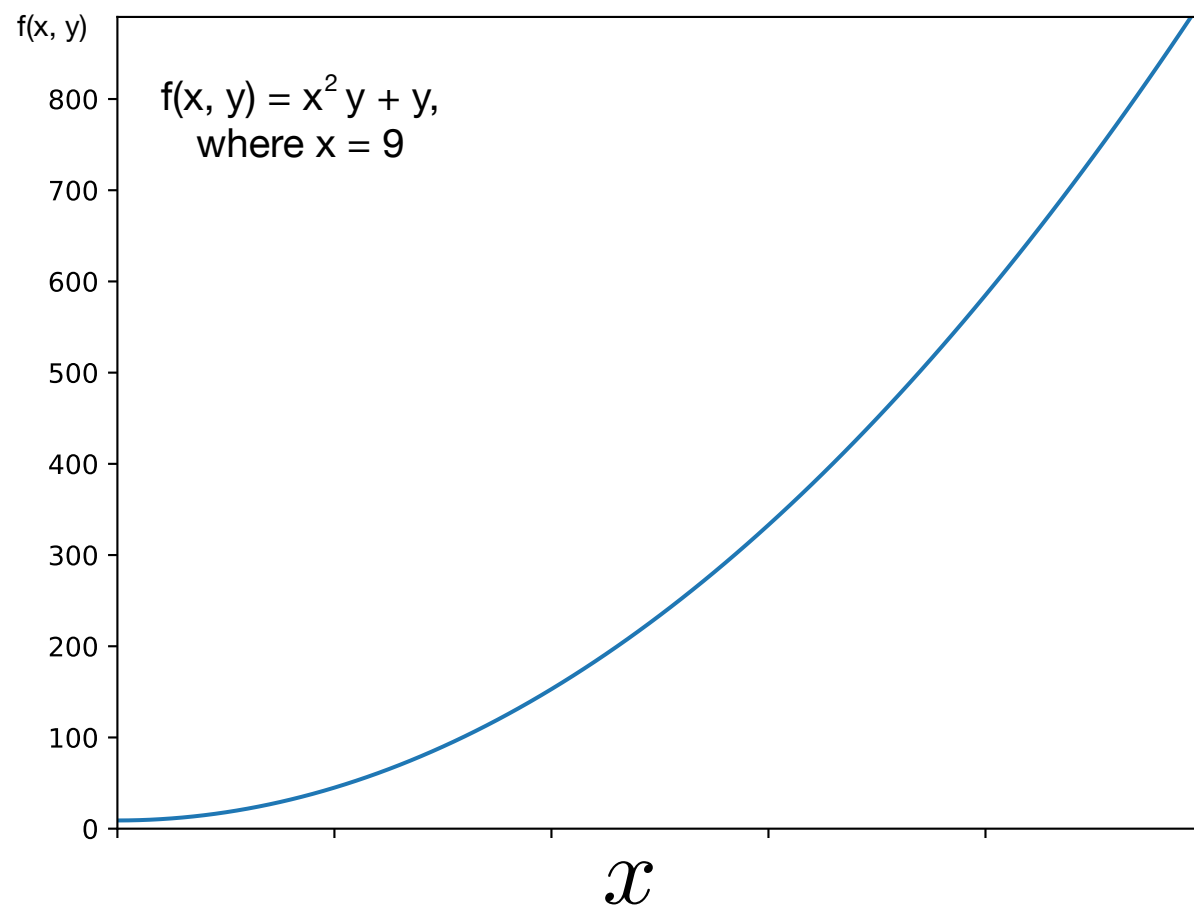
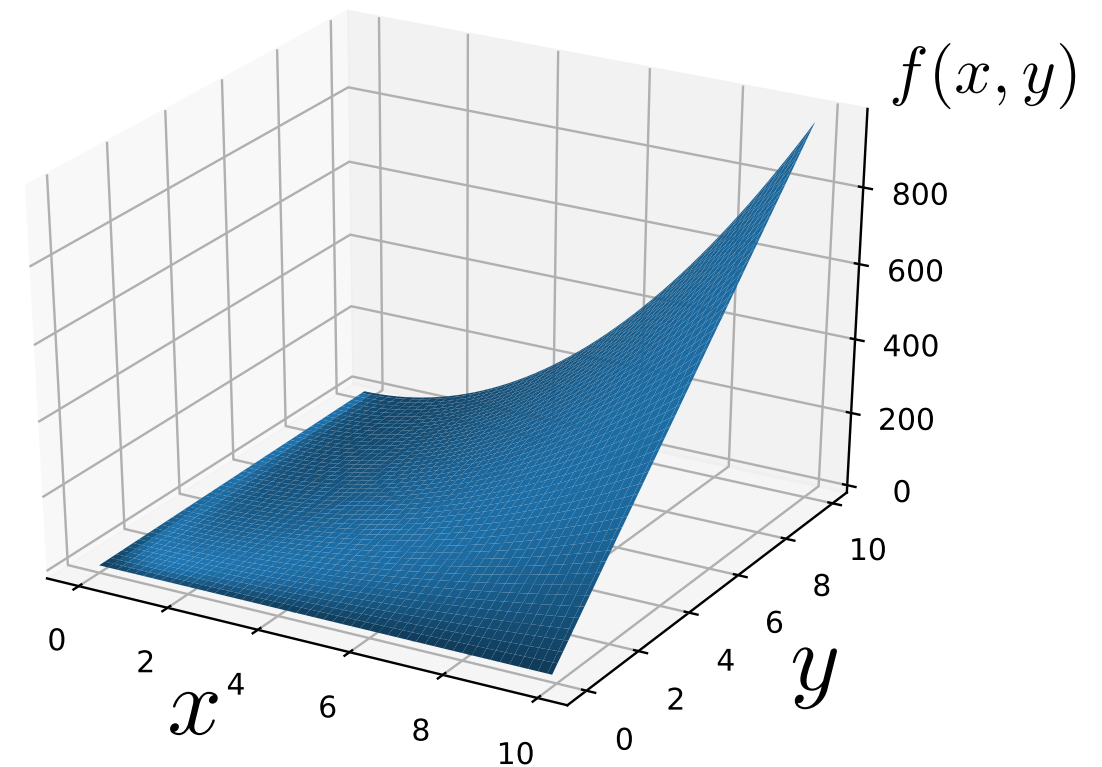
So, the gradient of the function f is defined as

$$\nabla f(x, y) = \begin{bmatrix} 2xy \\ x^2 + 1 \end{bmatrix}.$$

Gradients: Derivative of Multivariable Functions

Example: $f(x, y) = x^2 y + y$

$$\nabla f(x, y) = \begin{bmatrix} 2xy \\ x^2 + 1 \end{bmatrix}$$



Gradients & the Multivariable Chain Rule

Suppose we have a composite function like this:

$$f(g(x), h(x))$$

Remember the regular chain rule for a single input:

$$\frac{d}{dx} [f(g(x))] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

For two inputs, we now have

$$\frac{d}{dx} [f(g(x), h(x))] = \frac{\partial f}{\partial g} \cdot \frac{dg}{dx} + \frac{\partial f}{\partial h} \cdot \frac{dh}{dx}$$

Gradients & the Multivariable Chain Rule

$$f(g(x), h(x))$$

$$\frac{d}{dx} [f(g(x), h(x))] = \frac{\partial f}{\partial g} \cdot \frac{dg}{dx} + \frac{\partial f}{\partial h} \cdot \frac{dh}{dx}$$

Example:

$$f(g, h) = g^2 h + h$$

where $g(x) = 3x$, and $h(x) = x^2$

$$\frac{\partial f}{\partial g} = 2gh \qquad \frac{\partial f}{\partial h} = g^2 + 1$$

$$\frac{dg}{dx} = \frac{d}{dx} 3x = 3 \qquad \frac{dh}{dx} = \frac{d}{dx} x^2 = 2x$$

$$\begin{aligned} \frac{d}{dx} [f(g(x), h(x))] &= [2gh \cdot 3] + [(g^2 + 1) \cdot 2x] \\ &= 2xg^2 + 6gh + 2x \end{aligned}$$

Gradients & the Multivariable Chain Rule

in Vector Form

$$\begin{aligned} &f(g(x), h(x)) \\ \frac{d}{dx} [f(g(x), h(x))] &= \frac{\partial f}{\partial g} \cdot \frac{dg}{dx} + \frac{\partial f}{\partial h} \cdot \frac{dh}{dx} \\ &= \nabla f \cdot \mathbf{v}'(x). \end{aligned}$$

Where

$$\mathbf{v}(x) = \begin{bmatrix} g(x) \\ h(x) \end{bmatrix} \quad \mathbf{v}'(x) = \frac{d}{dx} \begin{bmatrix} g(x) \\ h(x) \end{bmatrix} = \begin{bmatrix} dg/dx \\ dh/dx \end{bmatrix}$$

Putting it together:

$$\nabla f \cdot \mathbf{v}'(x) = \begin{bmatrix} \partial f / \partial g \\ \partial f / \partial h \end{bmatrix} \cdot \begin{bmatrix} dg/dx \\ dh/dx \end{bmatrix} = \frac{\partial f}{\partial g} \cdot \frac{dg}{dx} + \frac{\partial f}{\partial h} \cdot \frac{dh}{dx}$$

The Jacobian (Matrix)

$$\mathbf{f}(x_1, x_2, \dots, x_m) = \begin{bmatrix} f_1(x_1, x_2, x_3, \dots, x_m) \\ f_2(x_1, x_2, x_3, \dots, x_m) \\ f_3(x_1, x_2, x_3, \dots, x_m) \\ \vdots \\ f_m(x_1, x_2, x_3, \dots, x_m) \end{bmatrix}$$

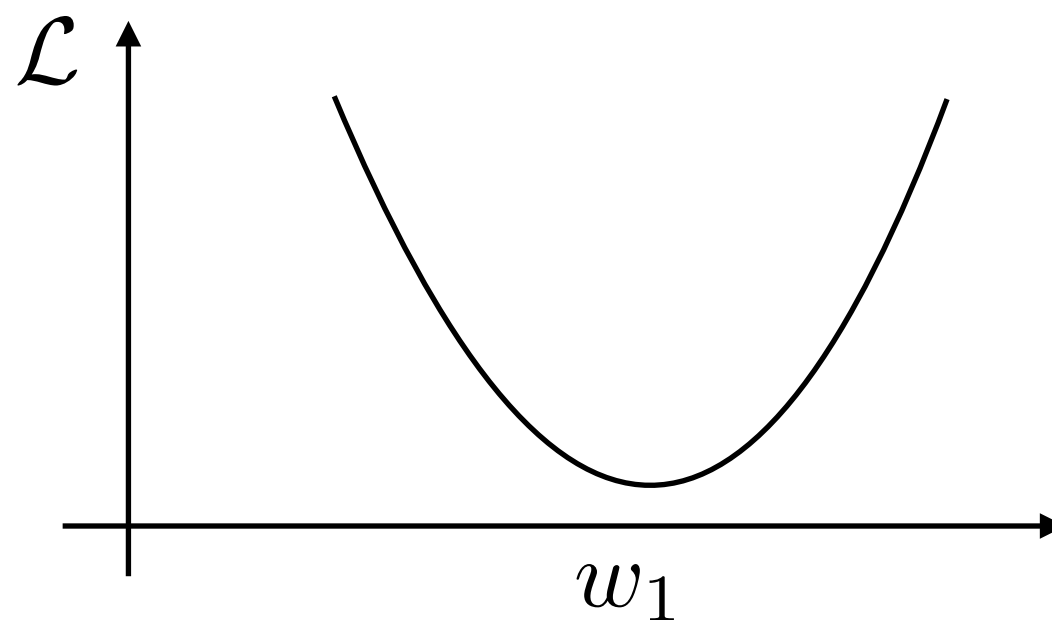
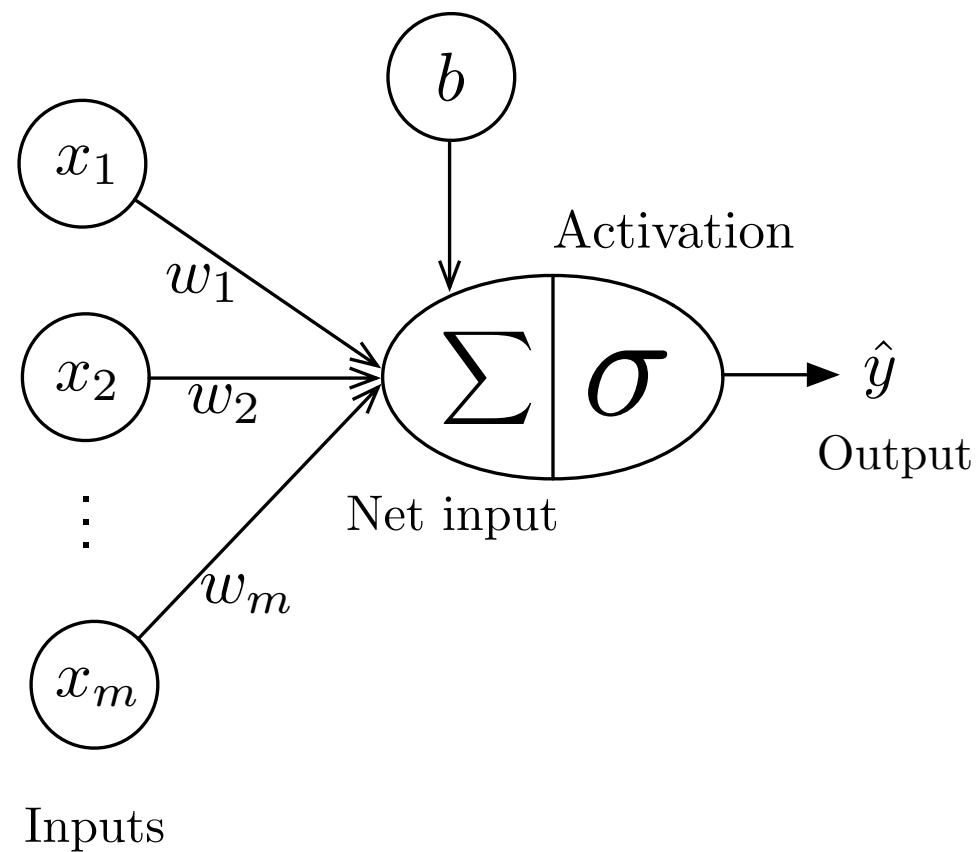
$$J(x_1, x_2, x_3, \dots, x_m) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \dots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \dots & \frac{\partial f_2}{\partial x_m} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} & \dots & \frac{\partial f_3}{\partial x_m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \frac{\partial f_m}{\partial x_3} & \dots & \frac{\partial f_m}{\partial x_m} \end{bmatrix}$$

The Jacobian (Matrix)

$$\mathbf{f}(x_1, x_2, \dots, x_m) = \begin{bmatrix} f_1(x_1, x_2, x_3, \dots, x_m) \\ f_2(x_1, x_2, x_3, \dots, x_m) \\ f_3(x_1, x_2, x_3, \dots, x_m) \\ \vdots \\ f_m(x_1, x_2, x_3, \dots, x_m) \end{bmatrix}$$

$$J(x_1, x_2, x_3, \dots, x_m) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} & \cdots & \frac{\partial f_3}{\partial x_m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \frac{\partial f_m}{\partial x_3} & \cdots & \frac{\partial f_m}{\partial x_m} \end{bmatrix} (\nabla f_1)^\top$$

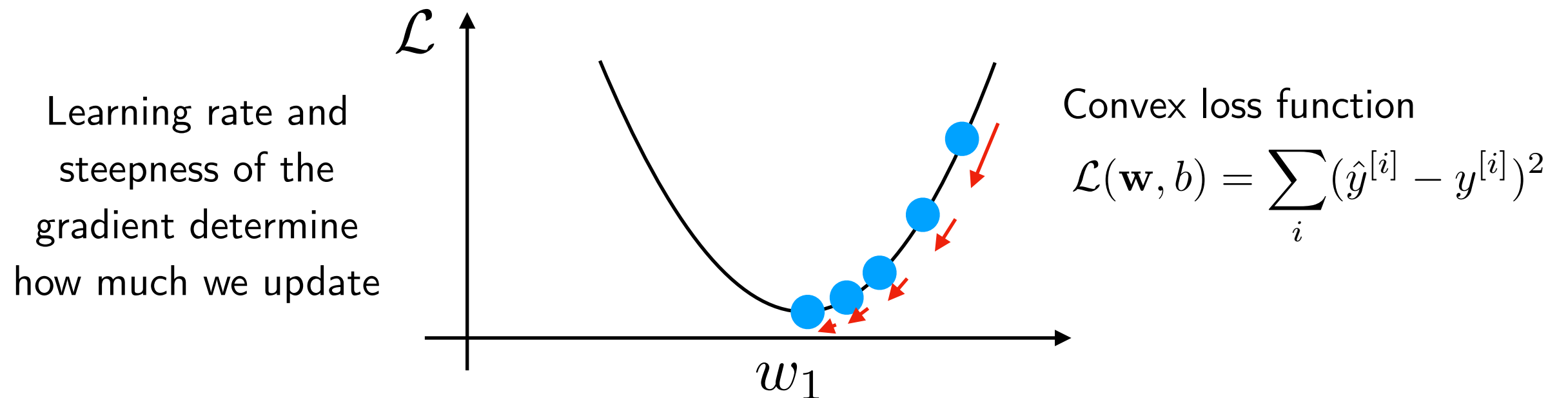
Back to Linear Regression



Convex loss function

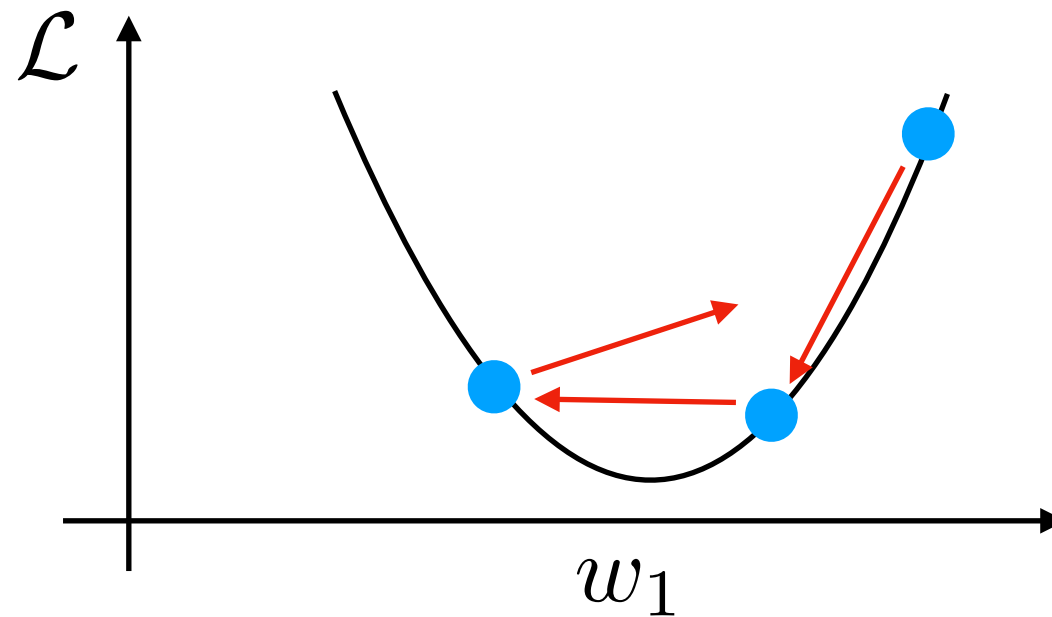
$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

Gradient Descent

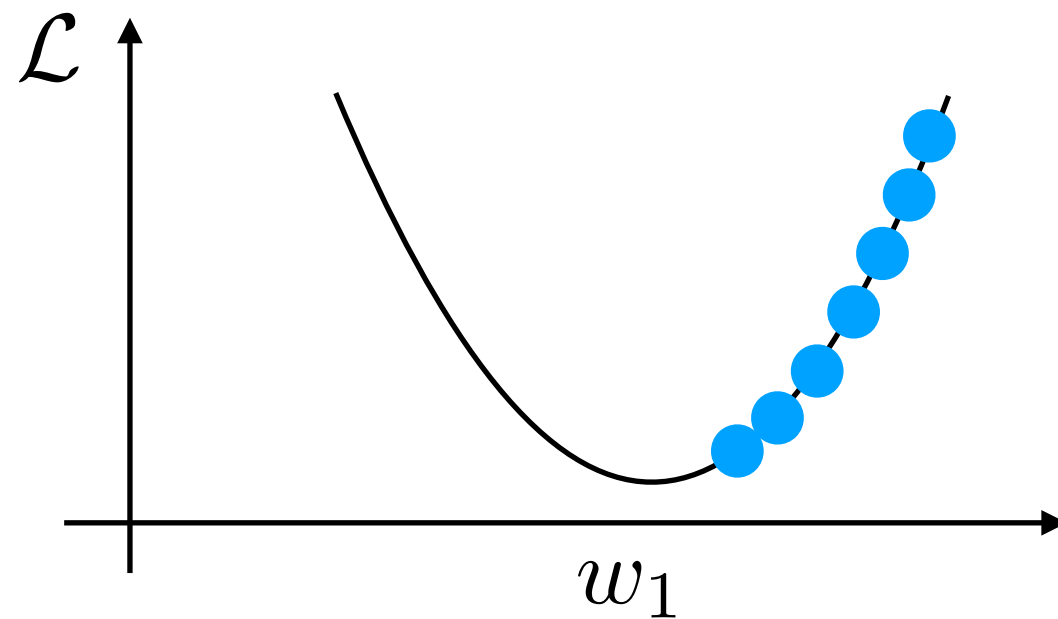


Gradient Descent

If the learning rate is too large,
we can overshoot



If the learning rate is too small,
convergence is very slow



Linear Regression Loss Derivative

$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \quad \text{Sum Squared Error (SSE) loss}$$

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

$$= \frac{\partial}{\partial w_j} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})^2$$

$$= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})$$

$$= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}^{[i]}$$

$$= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} x_j^{[i]} \quad \text{(Note that the activation function is the identity function in linear regression)}$$

$$= \sum_i 2(\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]}$$

Linear Regression Loss Derivative (alt.)

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

Mean Squared Error (MSE) loss often scaled by factor 1/2 for convenience

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

$$= \frac{\partial}{\partial w_j} \sum_i \frac{1}{2n} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})^2$$

$$= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]})$$

$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}^{[i]}$$

$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^T \mathbf{x}^{[i]})} x_j^{[i]}$$

(Note that the activation function is the identity function in linear regression)

$$= \frac{1}{n} \sum_i (\sigma(\mathbf{w}^T \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]}$$

Batch vs Stochastic

The minibatch and on-line modes are stochastic versions of gradient descent (batch mode)

Minibatch mode

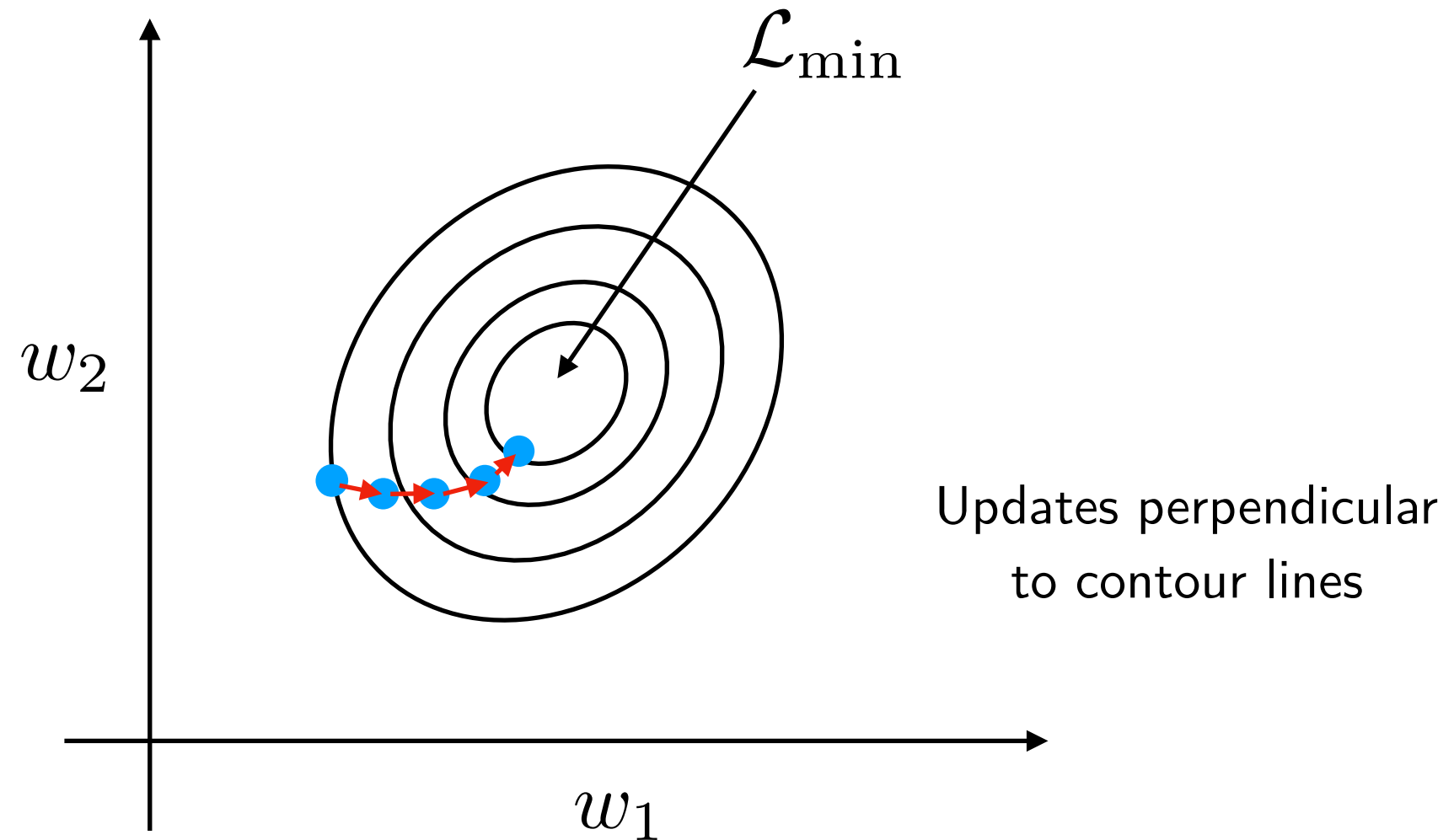
(mix between on-line and batch)

1. Initialize $\mathbf{w} := 0^{m-1}$, $\mathbf{b} := 0$
2. For every training epoch:
 - A. Initialize $\Delta \mathbf{w} := 0$, $\Delta b := 0$
 - B. For every $\{\langle \mathbf{x}^{[i]}, y^{[i]} \rangle, \dots, \langle \mathbf{x}^{[i+k]}, y^{[i+k]} \rangle\} \subset \mathcal{D}$:
 - (a) Compute output (prediction)
 - (b) Calculate error
 - (c) Update $\Delta \mathbf{w}, \Delta b$
 - C. Update $\mathbf{w}, b$:
$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, b := b + \Delta b$$

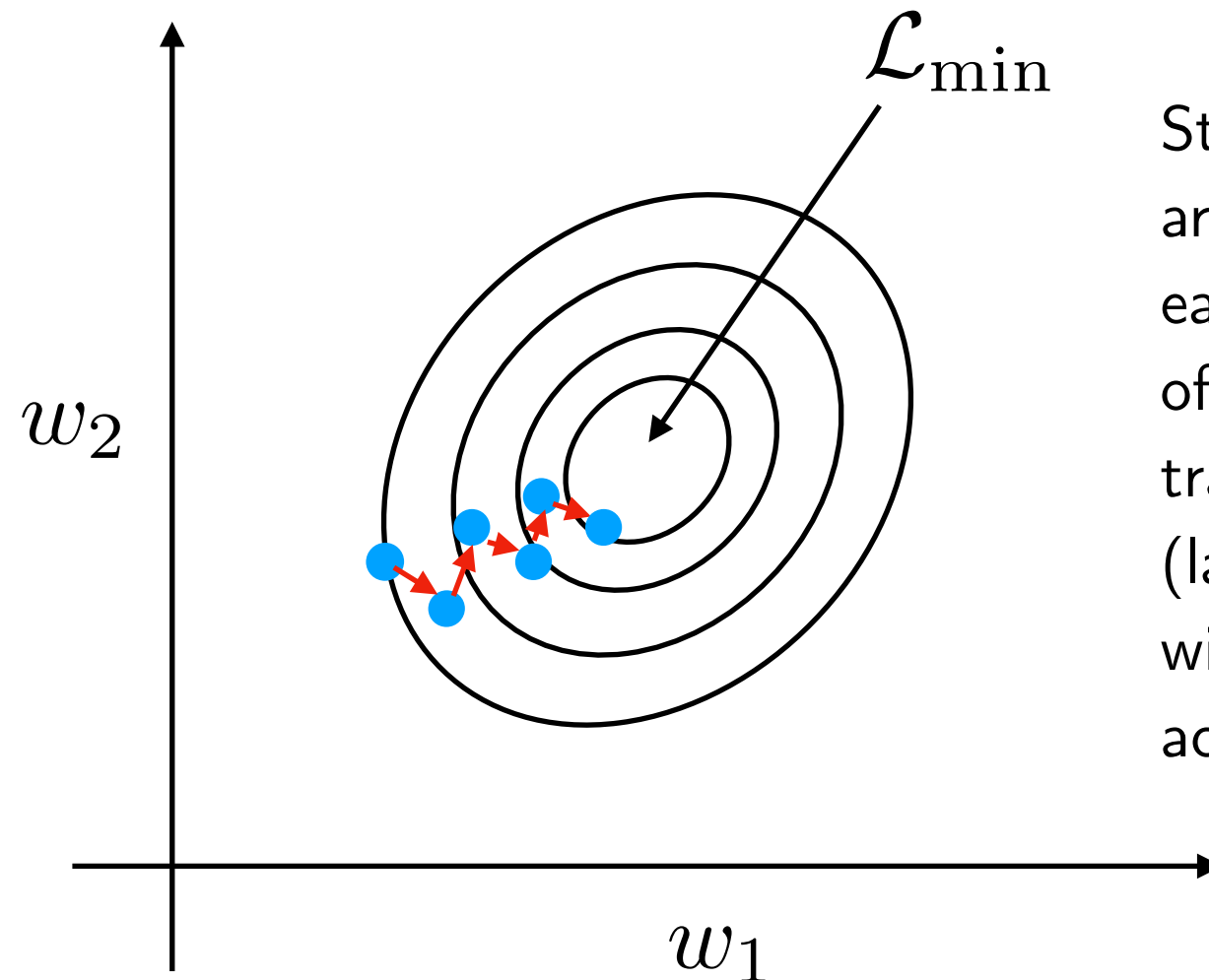
Most commonly used in DL, because

1. Choosing a subset (vs 1 example at a time) takes advantage of vectorization (faster iteration through epoch than on-line)
2. having fewer updates than "on-line" makes updates less noisy
3. makes more updates/epoch than "batch" and is thus faster

Batch Gradient Descent as Surface Plot

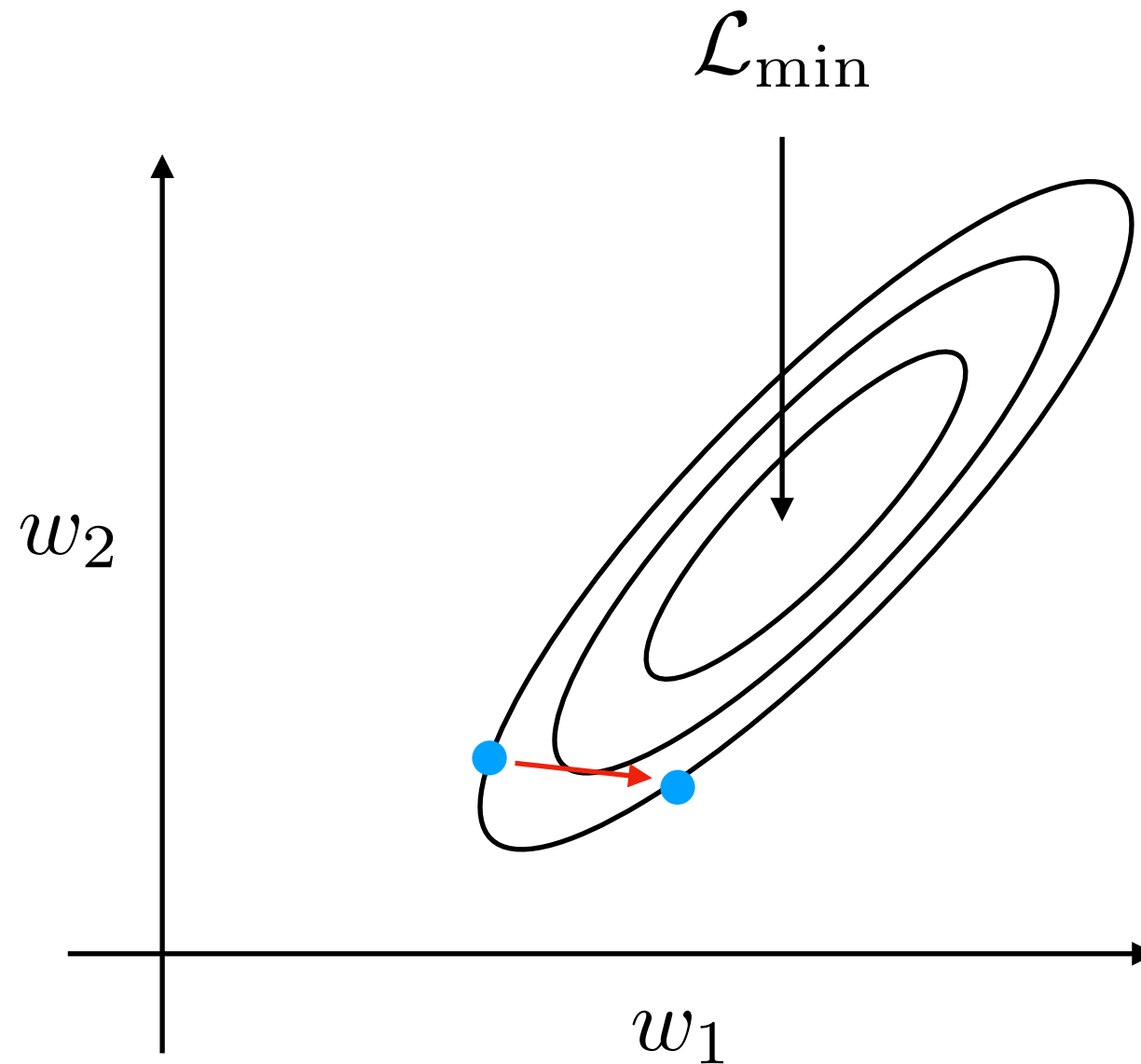


Stochastic Gradient Descent as Surface Plot



Stochastic updates are a bit noisier, because each batch is an approximation of the overall loss on the training set (later, in deep neural nets, we will see why noisier updates are actually helpful)

Batch Gradient Descent as Surface Plot



If inputs are on very different scales
some weights will update more than
others ... and it will also harm convergence
(always normalize inputs!)

Linear Regression

Code example:

https://github.com/rasbt/stat479-deep-learning-ss19/blob/master/L05_grad-descent/code/linear-regr-gd.ipynb